

THE SIGNAL

Issue #01

AI infrastructure for builders who ship

July 2026 · Free edition · payhip.com/b/4tHmb

Every week I scan the infrastructure layer — model releases, inference pricing shifts, open-source drops — and filter for what actually changes the math for builders. No hype cycles. No motivation-speak. Just the signal.

01 — Llama 3.3 70B hits GPT-4 class on most benchmarks

Meta's 70B model now matches or beats GPT-4o on MMLU, HumanEval, and MT-Bench at a fraction of the cost. Self-hosted inference on a Beelink mini-PC is no longer a compromise — it's a legitimate production path. The implication: the \$0/token local stack just became the default for most builder workloads.

02 — Groq inference drops below \$0.05/M input tokens for Llama

Groq's LPU-based inference is now pricing Llama 3.3 70B at \$0.05/M input, \$0.10/M output. For an agent running 500 tool calls/day at ~800 tokens each, that's under \$0.02/day in inference. The cost argument for keeping AI out of indie products just evaporated.

03 — Ollama 0.3 ships persistent context + structured output by default

Ollama's latest release adds persistent conversation context and enforced JSON schema output without any extra tooling. For local DAGR-style agents, this removes the two biggest friction points in building reliable tool-call loops. No cloud dependency required.

THE NUMBER

\$0.002

Cost per agent task on a self-hosted Llama 3.3 70B stack (Groq API, ~1,200 tokens/task)

In January 2024, running a GPT-4-class agent task cost roughly \$0.06–\$0.12. Eighteen months later, the same quality task on open-weight models via Groq costs \$0.002. That's a 30–60x cost reduction in under two years.

For a builder running a personal health pipeline, a social media agent, or a daily briefing loop: a full month of automated tasks at 100/day is under \$6. The economic barrier to personal AI infrastructure is effectively gone. The only remaining barrier is setup friction — which is exactly what the Agent Stack Kit solves.

The one-agent-one-task pattern beats monolithic assistants

Most people build AI agents wrong. They try to make one agent do everything — research, draft, post, schedule, notify. The agent gets confused, hallucinates tool calls, and fails in ways that are hard to debug.

The pattern that actually works at the infrastructure level is DAGR: Dispatch, Act, Gate, Report. Each agent has exactly one responsibility. Dispatch fires the right agent for the task. Act does the work. Gate checks the output. Report logs the result and triggers the next step.

Concrete implementation: On a Beelink mini-PC running Ollama, you run one model instance per agent role. Dispatch is a lightweight 7B model (fast, cheap). Act uses the 70B model (quality matters here). Gate is a 7B model with a structured output schema. The whole stack costs under \$300 in hardware and \$0/month in inference.

Before any agent touches an account, define its permission envelope, escalation path, and kill switch. The same one-agent-one-task discipline needs explicit gates around every tool.

Try it: Start with a single DAGR loop — one trigger (cron/webhook), one Act agent (Llama 3.3 70B via Groq), one Report step (Telegram message). Get that working before adding complexity.

THE SIGNAL

The Signal is a free weekly brief on AI infrastructure for builders and creators. Open source AI, inference cost trends, agent architecture patterns — filtered for people who ship, not people who spectate.

Want to go deeper?

Before an agent touches anything that matters, write the boundaries. The **AI Agent Permission & Governance Template: Secure Automation Kit** gives you four fill-in templates and a pre-launch checklist to define permissions, escalation paths, and a kill switch.

Get the Secure Automation Kit - \$19

payhip.com/b/4E5cH